



FaCENA
FACULTAD DE CIENCIAS EXACTAS
Y NATURALES Y AGRIMENSURA

Proyecto Integrador: **Sistema de Semáforos Inteligentes**

Materia: **Paradigmas y Lenguajes**

Grupo: 31



Facultad: Facultad de Ciencias Exactas y Naturales y Agrimensura (UNNE)

Fecha de Entrega: 16/06/2026



Índice

1. Introducción	4
1.1 Integrantes del grupo	5
1.2 Tema y contexto del proyecto	5
1.3 Descripción breve del sistema desarrollado	5
1.4 Alcance y límites del trabajo	5
2. Paradigma Funcional	5
2.1 Conceptos fundamentales aplicados	5
2.2 Funciones puras e impuras — clasificación	5
2.3 Inmutabilidad y ausencia de efectos secundarios	5
2.4 Funciones de orden superior	5
3. Fase 1 : Sistema de Semáforos en Common Lisp	5
3.1 Decisiones de diseño	5
3.2 Requerimiento 1 : transicion	5
3.3 Requerimiento 2 : semaforo-timer	5
3.4 Requerimiento 3 : log-cambio-estado	5
3.5 Requerimiento 4a : duracion-ciclo	5
3.6 Requerimiento 4b : recomendacion-ciclo	5
3.7 Requerimiento 5 : ciclos-por-tiempo	5
3.8 Requerimiento 6 : distribucion-temporal	5
3.9 Requerimiento 7 : ejemplos de uso y QA	5
3.10 Iteración 2 : Intermittencia de seguridad	5
3.11 Iteración 2 : Persistencia en archivo (informe)	6
4. Fase 2 : Ecosistema Quicklisp	6
4.1 Investigación de Quicklisp	6
4.2 Librería seleccionada : local-time	6
4.3 Justificación de la elección	6
4.4 Integración al sistema de auditoría	6
5. Fase 3 : Estudio Comparativo: Haskell	6
5.1 Presentación del lenguaje	6
5.2 Industrias y empresas que lo utilizan	6



5.3 Implementación de transicion en Haskell	6
5.4 Implementación de semaforo-timer en Haskell	6
5.5 Pregunta 1 : ADTs y tipado estático	6
5.6 Pregunta 2 : Lazy Evaluation	6
5.7 Conclusión del grupo sobre Haskell	6
6. Bitácora de Depuración	6
6.1 Bug 1 : Colisión de nombre timer con SBCL	6
6.2 Bug 2 : Problema de paquetes en el REPL	6
6.3 Bug 3 : floor retorna múltiples valores	6
6.4 Bug 4 : Constantes no evaluadas en lista literal	6
7. Uso de Inteligencia Artificial	7
7.1 Herramientas utilizadas	7
7.2 Partes del proceso donde se utilizó	7
7.3 Decisiones tomadas exclusivamente por el equipo	7
8. Conclusión	7
8.1 Resultados del proyecto	7
8.2 Aprendizajes del grupo	7
8.3 Mejoras futuras	7
9. Bibliografía	7
10. Anexo	7
Código fuente core.lisp	7
Código fuente solucion.hs	7
Capturas de ejecución	7



1. Introducción

1.1 Integrantes del grupo

Integrantes:

- Rodríguez Rivas Milton Nahuel
- Quintana Flores Lautaro Sebastián
- Nacimiento Néstor Javier
- Fernández Juliana Eva
- Rodríguez Adolfo Joaquín Jesús

1.2 Tema y contexto del proyecto

El presente trabajo práctico integrador fue desarrollado en el marco de la materia Paradigmas y Lenguajes, cursada durante el año 2026 en la Facultad de Ciencias Exactas y Naturales y Agrimensura (**FaCENA**) de la Universidad Nacional del Nordeste (**U.N.N.E**), correspondiente a la carrera Licenciatura en Sistemas de Información.

1.3 Descripción breve del sistema desarrollado

El proyecto consiste en el desarrollo de un sistema de semáforos inteligentes implementado en Common Lisp, con una comparativa adicional en Haskell. El sistema modela el comportamiento real de un semáforo urbano, aplicando estrictamente el paradigma de programación funcional: se implementaron funciones para modelar transiciones de estados (**rojo, amarillo, verde e intermitencia**), temporización automática basada en *timestamps Unix*, registro de auditoría con fechas legibles, análisis de ciclos semafóricos, planificación temporal y distribución porcentual de colores. Todas las funciones fueron diseñadas respetando los principios de **inmutabilidad** (Una vez que un dato es creado, no se puede modificar ni cambiar jamás), ausencia de efectos secundarios y composición funcional.



1.4 Alcance y límites del trabajo

El sistema cubre los seis requerimientos funcionales del enunciado base, más las dos extensiones de la Iteración 2, la integración de la librería **local-time** mediante **Quicklisp**, y la reescritura de las funciones **transición** y **semaforo-timer** en Haskell. El sistema no contempla interfaz gráfica, comunicación en red entre semáforos ni integración con hardware real. Su alcance es exclusivamente académico y orientado a la demostración del paradigma funcional.

2. Paradigma Funcional

2.1 Conceptos fundamentales aplicados

El paradigma funcional trata a la computación como la evaluación de funciones matemáticas. Sus pilares son la inmutabilidad (los datos no se modifican, son creados nuevos), las funciones puras (dada la misma entrada, siempre producen la misma salida sin efectos secundarios), y la composición (funciones complejas construidas a partir de funciones simples). Common Lisp, si bien es un lenguaje multiparadigma, permite aplicar estos principios de forma estricta mediante el uso de `defconstant`, recursividad y funciones de orden superior como `mapcar`.

2.2 Funciones puras e impuras: clasificación

Una función pura es aquella que, dada la misma entrada, produce siempre la misma salida y no genera efectos secundarios observables fuera de su ámbito — no modifica estructuras existentes, no escribe en pantalla ni en disco, no depende de estado externo.

En el sistema desarrollado, **transición** y **semaforo-timer** son los ejemplos más claros de funciones puras: **transición** evalúa dos símbolos y devuelve siempre la misma lista para la misma combinación de argumentos; **semaforo-timer** aplica `mod` sobre el timestamp y determina el color activo de forma completamente determinista. Ambas podrían ejecutarse mil veces con los mismos argumentos y producirían exactamente el mismo resultado.

En el extremo opuesto, **log-cambio-estado e informe** son funciones impuras: la primera escribe en la terminal (efecto secundario de I/O) y la segunda escribe en disco. Aunque ninguna modifica estructuras de datos en memoria, por lo que se clasifican como no destructivas. Su ejecución produce cambios observables fuera del programa.



2.3 Inmutabilidad y ausencia de efectos secundarios

En el sistema desarrollado se aplicó inmutabilidad absoluta: ninguna función modifica datos existentes. Los estados del semáforo no se almacenan en variables globales mutables, sino que fluyen como argumentos entre funciones. Las duraciones de cada color se definen como constantes mediante **defconstant**, que en Common Lisp garantiza que el valor no puede ser reasignado durante la ejecución. Se evitó deliberadamente el uso de **defparameter**, **defvar**, **setq** y **set f**, operadores que permitirían mutar estado, en cumplimiento estricto de las restricciones del paradigma. El resultado es un sistema donde el flujo de datos es completamente trazable: dado un timestamp de entrada, es posible reconstruir el estado exacto del semáforo en ese momento sin depender de ningún estado previo almacenado.

2.4 Funciones de orden superior

Las funciones de orden superior son aquellas que reciben funciones como argumentos o devuelven funciones como resultado. En Common Lisp, **mapcar** es la función de orden superior por excelencia: aplica una función a cada elemento de una lista y retorna una nueva lista con los resultados, sin modificar la original. En el sistema desarrollado, **distribución-temporal** utiliza **mapcar** con una función **lambda** para calcular el porcentaje de tiempo de cada color en una hora, transformando una lista de pares (**color. duración**) en una lista de pares (**color. porcentaje**). Esta decisión de diseño reemplaza un bucle imperativo por una transformación declarativa, expresando *qué* se quiere calcular en lugar de como iterarlo paso a paso.

3. Fase 1: Sistema de Semáforos en Common Lisp

3.1 Decisiones de diseño

La primera decisión de diseño relevante fue la representación de los estados del semáforo mediante símbolos (**'en-rojo**, **'en-amarillo**, **'en-verde**) en lugar de strings o enteros. Esta elección responde a varios criterios: en primer lugar, el propio enunciado especifica símbolos como tipo de entrada y salida; en segundo lugar, los símbolos en Common Lisp son objetos únicos e internados en memoria, lo que permite compararlos con **eq** (comparación por identidad de dirección de memoria) en lugar de **equal** (comparación por contenido), resultando más eficiente. Finalmente, los símbolos son extensibles — un sistema externo que controle hardware **real** podría interpretarlos directamente como identificadores de estado, de forma análoga a un Algebraic Data Type en Haskell.

La segunda decisión fue centralizar las duraciones de cada color en constantes mediante **defconstant**. Escribir los valores numéricos directamente en el código (números mágicos) implicaría repetirlos en múltiples funciones, haciendo cualquier modificación futura propensa a



errores e inconsistencias. Con **defconstant**, un cambio en la duración del ciclo se realiza en un único lugar y se propaga automáticamente a todo el sistema. Adicionalmente, **defconstant** es semánticamente correcto en el paradigma funcional: declara valores que no cambian durante la ejecución, sin violar la restricción de inmutabilidad.

3.2 Requerimiento 1: transición

La función **transición** recibe dos símbolos: el color actual del semáforo (**en-rojo**, **en-amarillo**, **en-verde**) y el color destino (**rojo**, **amarillo**, **verde**), y devuelve una lista con el estado actual y una acción como string ("**cambiar-a-verde**"). Se implementó mediante **cond** evaluando cada combinación válida con **eq**. Las transiciones válidas son exactamente tres: rojo → verde, verde → amarillo, amarillo → rojo. Cualquier otra combinación cae en el caso por defecto retornando el símbolo '**acción-por-defecto**'. Se eligió **eq** sobre **equalp** porque los símbolos son objetos únicos en memoria y la comparación por identidad es más eficiente que la comparación por contenido.

En la Iteración 2 se extendió **transición** para contemplar el estado de intermitencia. Se agregaron cuatro casos nuevos: **en-rojo** → **intermitencia**, **en-intermitencia** → **verde**, **en-verde** → **intermitencia** y **en-intermitencia** → **amarillo**, manteniendo los tres casos originales para preservar compatibilidad con transiciones directas. El estado '**en-intermitencia**' actúa como estado intermedio obligatorio entre los cambios de luz principal.

3.3 Requerimiento 2: semaforo-timer

La función **semaforo-timer** recibe un timestamp Unix (entero) y devuelve el color activo en ese momento. El ciclo semafórico tiene una duración total de 222 segundos (incluyendo intermitencia). Mediante **mod** se obtiene la posición dentro del ciclo actual: un offset entre 0 y 221, y con **cond** se determina el color correspondiente según los rangos: 0-89 rojo, 90-92 intermitencia, 93-212 verde, 213-215 intermitencia, 216-221 amarillo. El uso de **let** para calcular el offset una sola vez evita recalcular **mod** en cada rama del **cond**, mejorando la eficiencia sin violar inmutabilidad. Cabe destacar que **let** define una variable local de solo lectura.

3.4 Requerimiento 3: log-cambio-estado

La función **log-cambio-estado** recibe un *epoch*, un color anterior y un color nuevo, e imprime en la terminal un registro del cambio de estado. Es la única función clasificada como impura entre los requerimientos principales, dado que su propósito es producir un efecto secundario de I/O. Se integró la librería **local-time** (Fase 2) para convertir el timestamp Unix a formato legible (YYYY-MM-DD HH:MM: SS) mediante **local-time: unix-to-timestamp** y **local-time: format-**



timestring. La función acepta cualquier tipo en los argumentos de color por diseño deliberado: **format ~a** serializa cualquier objeto Lisp, lo que permite registrar tanto símbolos como strings sin crashear en la demo.

3.5 Requerimiento 4a: duración-ciclo

La función **duración-ciclo** recibe una cantidad de segundos y devuelve una lista con dos elementos: la cantidad de ciclos completos que caben en ese tiempo (calculado con **floor**) y una recomendación sobre la duración del ciclo actual. La recomendación se obtiene llamando a **recomendación-ciclo** con **+duración-ciclo-total+**. Es decir, evalúa si el ciclo configurado actualmente (222 segundos) está dentro del rango psicológicamente óptimo para conductores.

Se usó **nth-value 0** para extraer únicamente el **cociente** de **floor**, la cantidad de ciclos completos, descartando el resto que representa los segundos sobrantes que no completan un ciclo adicional.

3.6 Requerimiento 4b: recomendación-ciclo

La función **recomendación-ciclo** recibe una duración en segundos y devuelve un símbolo descriptivo según el rango: **ciclo-corto** para duraciones menores a 35 segundos, **ciclo-óptimo** para el rango 35-150 segundos, y **ciclo-largo** para duraciones mayores a 150 segundos. Estos rangos responden a criterios de psicología del conductor establecidos en el enunciado. Con la configuración actual de 222 segundos, el sistema retorna **ciclo-largo**, dato que el sistema reporta correctamente como información para el equipo de ingeniería de tráfico.

3.7 Requerimiento 5: ciclos-por-tiempo

La función **ciclos-por-tiempo** recibe una duración en minutos y devuelve la cantidad de ciclos completos que se realizan en ese período. La implementación convierte los minutos a segundos multiplicando por 60 y aplica **floor** para obtener la división entera contra **+duración-ciclo-total+**. Es la función más concisa del sistema, una sola expresión aritmética, y un ejemplo claro de función pura: determinista, sin efectos secundarios y no destructiva.

3.8 Requerimiento 6: distribución-temporal

La función **distribución-temporal** calcula el porcentaje de tiempo que ocupa cada color en una hora. Se implementó con **mapcar** sobre una lista de pares (**color. duración**), aplicando una función **lambda** que calcula el porcentaje de cada color en base a los ciclos completos por hora. Es la función más representativa del paradigma funcional en el sistema: reemplaza completamente un bucle imperativo por una transformación declarativa, cumpliendo la restricción de cero bucles imperativos y demostrando el uso de funciones de orden superior.



3.9 Requerimiento 7: ejemplos de uso y QA

La función **run-ejemplos** ejecuta una batería de casos de prueba para cada requerimiento, cubriendo tres escenarios por función: camino normal (comportamiento esperado), camino alternativo (casos bordes o transiciones inválidas) y camino de error (argumentos de tipo incorrecto). El sistema maneja los errores de forma silenciosa retornando **NIL** o **'acción- defecto por-** en lugar de crashear, decisión tomada deliberadamente para garantizar estabilidad en la demo en vivo.

3.10 Iteración 2: Intermitencia de seguridad

En la segunda iteración se incorporó un estado de intermitencia de 3 segundos entre cada cambio de luz principal, modelado mediante el símbolo **'intermitencia**. Esta extensión implicó tres modificaciones al sistema base: primero, se agregó la constante **+duración-intermitencia+** con valor 3 y se actualizó **+duración-ciclo-total+** de 216 a 222 segundos para reflejar los dos períodos de intermitencia por ciclo. Segundo, se extendió **transición** con cuatro casos nuevos que contemplan las transiciones hacia y desde el estado intermitente. Tercero, se actualizaron los rangos de **semaforo-timer** para reflejar el nuevo ciclo: rojo (0-89), intermitencia (90-92), verde (93-212), intermitencia (213-215), amarillo (216-221). La función **distribución-temporal** también fue actualizada para incluir el porcentaje de tiempo correspondiente a la intermitencia, calculado como el doble de **+duración-intermitencia+** dado que ocurre dos veces por ciclo.

3.11 Iteración 2: Persistencia en archivo (informe)

Se implementó la función **informe** para registrar en un archivo de texto plano (**informe-ejecucion-semaforo.txt**) el historial de cambios de estado del semáforo. La función recibe una lista de datos donde cada elemento es una terna (**epoch color-anterior color-nuevo**), escribe un encabezado y luego itera sobre los datos mediante **mapcar**, cumpliendo la restricción de cero bucles imperativos. Para cada elemento convierte el epoch a fecha legible usando **local-time**, produciendo líneas con el formato **YYYY-MM-DD HH:MM:SS - Transición: COLOR → COLOR**. La función se clasifica como impura por su efecto secundario de escritura en disco, y no destructiva porque no modifica ninguna estructura de datos en memoria.



4. Fase 2: Ecosistema Quicklisp

4.1 Investigación de Quicklisp

Quicklisp es el gestor de paquetes estándar de facto para Common Lisp, disponible en quicklisp.org. Funciona de forma similar a **pip** en Python o **npm** en Node.js: permite instalar, cargar y gestionar librerías de terceros con una interfaz simple. Una vez instalado, cargar una librería es tan simple como ejecutar (**ql: quickload: nombre-libreria**) en el REPL, lo que descarga automáticamente la librería y sus dependencias desde el repositorio central de Quicklisp. A diferencia de otros gestores, Quicklisp utiliza *snapshot* mensuales del ecosistema de librerías, garantizando que todas las dependencias de un *Snapshot* sean compatibles entre sí.

4.2 Librería seleccionada: local-time

De las dos opciones disponibles en la consigna (**local-time** y **cl-json**), el grupo seleccionó **local-time**.

4.3 Justificación de la elección

Se eligió **local-time** por dos razones principales. Primero, su integración es más visual y directamente verificable en la demo: el cambio de un timestamp Unix crudo como **200000** a una fecha legible como **1970-01-03 07:33:20** es inmediatamente evidente en la terminal sin necesidad de explicación adicional. Segundo, **local-time** se integra de forma más natural al sistema de auditoría ya existente. Modifica una función ya implementada en lugar de requerir una nueva capa de carga de configuración externa.

4.4 Integración al sistema de auditoría

La integración de **local-time** se realizó modificando la función **log-cambio-estado** y la función **informe**. En ambos casos se reemplazó la impresión directa del epoch por una conversión en dos pasos: **local-time: unix-to-timestamp** convierte el entero Unix a un objeto timestamp interno de la librería, y **local-time: format-timestring** formatea ese objeto al string legible con el patrón **YYYY-MM-DD HH:MM: SS**. La carga de la librería se realiza al inicio de la sesión con (**ql: quickload: local-time**), sin modificar la estructura del archivo **core.lisp**.



5. Fase 3: Estudio Comparativo: Haskell

5.1 Presentación del lenguaje

Haskell es un lenguaje de programación puramente funcional, de tipado estático fuerte con inferencia de tipos, y evaluación perezosa (lazy evaluation). Fue diseñado en 1990 por un comité académico con el objetivo de crear un lenguaje funcional estándar y abierto. A diferencia de Common Lisp, que es multiparadigma, Haskell es funcional puro, no existe la mutabilidad de estado y toda función es pura por defecto. Su sistema de tipos es considerado uno de los más expresivos y seguros entre los lenguajes de uso general, permitiendo al compilador detectar clases enteras de errores antes de ejecutar el programa.

5.2 Industrias y empresas que lo utilizan

Haskell tiene presencia destacada en industrias donde la corrección del software es crítica. En el sector financiero, Standard Chartered y Facebook (Meta) lo utilizan para sistemas de análisis y detección de spam respectivamente. GitHub lo usa en su motor de detección semántica de código. IOHK, la empresa detrás de la blockchain Cardano, desarrolla toda su infraestructura en Haskell. En el ámbito académico y de investigación es ampliamente usado como lenguaje de referencia para el estudio de sistemas de tipos y semántica de lenguajes. También tiene presencia en compiladores, de hecho, GHC, el compilador principal de Haskell, está escrito en el propio Haskell.



5.3 Implementación de transición en Haskell

```
1  -- -----
2  -- FUNCIÓN: transicion
3  -- NATURALEZA: Pura
4  -- ESTRATEGIA: Pattern Matching
5  -- IMPACTO: No destructiva
6  -- -----
7  transicion :: Color -> String -> (Color, String)
8  transicion EnRojo "verde"      = (EnRojo, "cambiar-a-verde")
9  transicion EnVerde "amarillo" = (EnVerde, "cambiar-a-amarillo")
10 transicion EnAmarillo "rojo"  = (EnAmarillo, "cambiar-a-rojo")
11 transicion color _            = (color, "accion-por-defecto")
12
13
14
```

La re implementación de **transición** en Haskell utiliza *pattern matching* directo en los argumentos de la función, donde cada línea es un caso independiente que el compilador evalúa en orden. La firma **Color -> String -> (Color, String)** declara explícitamente los tipos: recibe un **Color** y un **String**, y devuelve una **tupla** con ambos. El caso por defecto **transición color _** usa (guion bajo) **_** para ignorar el segundo argumento, equivalente al **t** del **cond** en Lisp. A diferencia de Common Lisp donde los estados son símbolos sin restricción de tipo, en Haskell el compilador garantiza que el primer argumento solo puede ser **EnRojo**, **EnAmarillo** o **EnVerde**. En cualquier otro valor es un error de compilación.



5.4 Implementación de semaforo-timer en Haskell

```
1  -- -----  
2  -- FUNCIÓN: semaforoTimer  
3  -- NATURALEZA: Pura  
4  -- ESTRATEGIA: Guardas (Guard Expressions)  
5  --             evaluación condicional sobre el argumento  
6  -- IMPACTO: No destructiva  
7  -- -----  
8  semaforoTimer :: Int -> Color  
9  semaforoTimer timestamp  
10 | offset <= 89 = EnRojo  
11 | offset <= 95 = EnAmarillo  
12 | otherwise   = EnVerde  
13 where offset = mod timestamp 216  
14
```

La función **semaforoTimer** se implementó mediante guardas, una construcción de Haskell que evalúa condiciones booleanas en orden, equivalente al **cond** de Lisp. La cláusula **where** define **offset** como variable local de solo lectura, calculando **mod** una única vez y reutilizándolo en todas las guardas. **otherwise** es el caso por defecto, análogo al **t** en Lisp. La firma **Int -> Color** explicita que la función recibe un entero y devuelve un **Color**, donde el compilador verifica esto en tiempo de compilación.

5.5 Pregunta 1: ADTs y tipado estático

Un Algebraic Data Type (ADT) es una forma de definir tipos de datos enumerando explícitamente todos sus valores posibles. En el sistema desarrollado se definió:

```
1 data Color = EnRojo | EnAmarillo | EnVerde deriving (Show)
```

Esto le indica al compilador que **Color** puede ser exactamente una de esas tres variantes y ninguna más. El operador **|** representa una disyunción: **Color** es **EnRojo** o **EnAmarillo** o **EnVerde**. **deriving (Show)** instruye al compilador a generar automáticamente una representación imprimible del tipo.

La consecuencia directa, es que pasar un argumento de tipo incorrecto a **transición**, por ejemplo, un entero o un **String** donde se espera un **Color**, indicando un error de compilación antes de



ejecutar el programa. El compilador rechaza el código. Esto contrasta con Common Lisp, donde los estados son símbolos sin restricción de tipo: es perfectamente posible llamar **(transición 'en-violeta 'verde)** y el programa compila y ejecuta, cayendo en **acción-por-defecto**. En Haskell ese estado inválido no puede existir en tiempo de ejecución porque el compilador lo impide en tiempo de compilación.

5.6 Pregunta 2: Lazy Evaluation

La evaluación perezosa (lazy evaluation) es una estrategia de evaluación en la cual las expresiones no se computan hasta que su valor es estrictamente necesario. Haskell la implementa por defecto en todo el lenguaje, a diferencia de la mayoría de los lenguajes, incluyendo Common Lisp, que usan evaluación estricta (eager evaluation), donde los argumentos se evalúan antes de pasarlos a una función.

Una consecuencia directa de la lazy evaluation es que Haskell puede trabajar con estructuras de datos conceptualmente infinitas sin agotar la memoria:

```
1 timestamps = [0..] -- lista infinita de timestamps
2 colores = map semaforoTimer timestamps -- infinita lista de colores
```

En un lenguaje estricto este código intentaría generar infinitos elementos inmediatamente, agotando la memoria. En Haskell no ocurre nada hasta que alguien solicita elementos concretos:

```
1 take 10 colores -- calcula exactamente 10 colores, nada más
```

Aplicado al sistema de semáforos: si existiera un flujo continuo de eventos de tráfico (un timestamp por segundo durante años), Haskell permitiría modelarlo como una lista infinita y procesar únicamente los eventos requeridos en cada momento, sin cargar el conjunto completo en memoria. El sistema consumiría recursos proporcionales a lo que se necesita procesar, no al tamaño total del flujo.

Esta característica es particularmente valiosa en sistemas embebidos de tiempo real como el descrito en el enunciado, donde el flujo de eventos es potencialmente ilimitado y la memoria disponible es restringida.



5.7 Conclusión del grupo sobre Haskell

El aspecto más sorprendente de Haskell en comparación con Common Lisp es la rigidez expresiva de su sistema de tipos. En Lisp todo es fluctuante, puesto que un símbolo puede pasarse donde se espera cualquier cosa, y el programa decide en tiempo de ejecución qué hacer con ello. Haskell invierte completamente esa filosofía: cada valor, cada argumento y cada retorno debe pertenecer a un tipo definido explícitamente, y el compilador verifica eso antes de ejecutar una sola línea. Definir **data Color = EnRojo | EnAmarillo | EnVerde** no es solo documentación, sino que es una restricción real que el compilador hace cumplir. Un estado inválido como **EnVioleta** simplemente no puede existir en tiempo de ejecución.

La segunda característica destacable es la lazy evaluation, que permite trabajar con estructuras de datos de tamaño arbitrario o incluso infinito sin comprometer la memoria. Mientras que en Lisp procesar una lista muy grande implica tenerla en memoria, Haskell calcula cada elemento solo cuando se lo necesita, en consecuencia, lo que lo hace especialmente adecuado para sistemas de tiempo real con flujos continuos de eventos, como el sistema semafórico descrito en este trabajo.

Como conclusión general, el grupo rescata que Haskell obliga a pensar el problema en términos de tipos y transformaciones antes de escribir una línea de código, lo que resulta en programas más predecibles y seguros. La curva de aprendizaje es más pronunciada que la de Lisp, pero las garantías que ofrece el compilador compensan ese esfuerzo en sistemas donde la corrección es crítica.

6. Bitácora de Depuración

6.1 Bug 1: Colisión de nombre timer con SBCL

Descripción: Al intentar definir la función principal del temporizador con el nombre **timer**, SBCL generaba advertencias o comportamiento inesperado debido a que ese nombre colisiona con una clase interna del sistema de timers de SBCL.

Código que generó el error:

```
example.lisp > ...  
1 (defun timer (timestamp)  
2   ...)
```




```
Debug x
Condition
Lock on package SB-EXT violated when proclaiming TIMER as a function
while in
package COMMON-LISP-USER.
See also:
The SBCL Manual, Node "Package Locks"

Restarts
0: [continue] Ignore the package lock.
1: [ignoreAll] Ignore all package locks in the context of this
operation.
2: [unlockPackage] Unlock the package.
3: [abort] abort thread (#<THREAD tid=131193 "169 - $/alive/eval"
RUNNING {100450B033}>)

Backtrace
0 | PACKAGE-LOCK-VIOLATION
  | SYS:SRC;CODE;TARGET-PACKAGE.LISP
  | >6 Local variables
1 | ASSERT-SYMBOL-HOME-PACKAGE-UNLOCKED
  | SYS:SRC;CODE;TARGET-PACKAGE.LISP
  | >3 Local variables
2 | PROCLAIM-TO-FUN-NAME
```

Causa: SBCL utiliza el símbolo **timer** internamente en sus paquetes. Al definir una función con ese nombre en: **cl-user**, se produce una colisión de namespace que genera advertencias o redefinición de comportamiento inesperado.

Solución: Se renombró la función a **semaforo-timer**, prefijando con el dominio del problema para evitar cualquier colisión con símbolos del sistema. Este tipo de conflicto es un ejemplo concreto de por qué los sistemas de paquetes existen en Common Lisp.

6.2 Bug 2: Problema de paquetes en el REPL

Descripción: Al intentar organizar el código en un paquete **:semaforo** para resolver el problema anterior, el REPL dejó de reconocer las funciones definidas en el archivo.

Código que generó el error:

```
15 | (defpackage :semaforo
16 |   (:use :cl)
17 |   (:export #:transicion ...))
18 | (in-package :semaforo)
19 |
```




Al llamar desde el REPL:

```
cl-user> (transicion 'en-rojo 'verde)
```

El error obtenido fue:

The function COMMON-LISP-USER::TRANSICION is undefined. Restarts 0: [continue] Retry using TRANSICION. 1: [useValue] Use specified function 2: [abort] abort thread

Causa: El REPL del plugin Alive para VS Code opera en el paquete `:cl-user` por defecto y no respeta el `in-package` evaluado manualmente. Las funciones quedaban definidas en `:semaforo`, pero se llamaban desde `:cl-user`, donde no existían.

Solución: Se descartó el uso de paquetes por ser innecesario para el alcance del TP. Se eliminó el `defpackage` y el `in-package`, y todas las funciones quedaron en `:cl-user`. El archivo se carga completo con `(load "core.lisp")` desde el REPL para garantizar que todas las definiciones estén disponibles en el paquete activo.

6.3 Bug 3: floor retorna múltiples valores

Descripción: Al usar `floor` dentro de `list` para construir el resultado de `duracion-ciclo`, la lista resultante tenía un elemento extra inesperado.

Código que generó el error:

```
temporary.lisp > ...
1 (defun duracion-ciclo (segundos)
2   (list (floor segundos +duracion-ciclo-total+)
3         (recomendacion-ciclo +duracion-ciclo-total+)))
4
```

```
cl-user> (duracion-ciclo 4234)
(19 CICLO-LARGO)

cl-user> (mod 400 222)
178

cl-user> (floor 400 222)
1
178
cl-user>
```



6.4 Bug 4: Constantes no evaluadas en lista literal

Causa: En Common Lisp, **floor** es una función que retorna múltiples valores, es decir el cociente y el resto de la división entera. Al pasarlo directamente dentro de **list**, ambos valores se insertaron como elementos separados, produciendo una lista de tres elementos en lugar de dos.

Solución: Se utilizó **nth-value 0** para extraer únicamente el primer valor retornado por **floor**, es decir el cociente, y descartando el resto:

```
(defun duracion-ciclo (segundos)
  (cond
    ((integerp segundos)
     (list (nth-value 0 (floor segundos +duracion-ciclo-total+))
           (recomendacion-ciclo +duracion-ciclo-total+)))
    (t
     )))
```

7. Uso de Inteligencia Artificial

7.1 Herramientas utilizadas

Durante el desarrollo del proyecto se utilizó Claude (Anthropic) como herramienta de asistencia. No se utilizaron herramientas de generación automática de código, como GitHub Copilot, ni se generaron bloques completos de código mediante prompts directos al enunciado.

7.2 Partes del proceso donde se utilizó

La IA fue utilizada en modalidad co-piloto a lo largo del desarrollo: como tutor dinámico para explicar conceptos del paradigma funcional (símbolos vs strings, **eq** vs **equalp**, múltiples valores de **floor**, lazy evaluation en Haskell), como detector de errores sintácticos y conceptuales durante la implementación de cada función, y como asistente de redacción para el presente informe ,



donde el contenido y las conclusiones son propios del grupo y la IA colaboró en la estructuración y redacción formal.

La lógica de cada función fue diseñada, escrita y corregida por el grupo. En ningún caso se proporcionó el enunciado completo a la IA solicitando una solución automática.

7.3 Decisiones tomadas exclusivamente por el equipo

La decisión técnica más relevante tomada de forma autónoma fue la incorporación de validación de tipos mediante **integerp** en las funciones que reciben timestamps y duraciones numéricas. Esta decisión no fue sugerida externamente sino adoptada por el grupo como medida de robustez para garantizar comportamiento predecible durante la demo en vivo, ya que se priorizó que el sistema retorne **NIL** silenciosamente ante argumentos inválidos en lugar de lanzar un error en pantalla. También fue decisión del grupo utilizar **defconstant** en lugar de números directamente en el código, y renombrar **timer** a **semaforo-timer** para evitar colisiones de namespace con SBCL.

8. Conclusión

8.1 Resultados del proyecto

Se implementó exitosamente el sistema de semáforos inteligentes en Common Lisp, cubriendo la totalidad de los requerimientos funcionales (1 al 6), ambas extensiones de la Iteración 2, la integración de la librería **local-time** mediante Quicklisp, y la re implementación comparativa de **transicion** y **semaforoTimer** en Haskell. El sistema respeta estrictamente las restricciones del paradigma funcional: inmutabilidad absoluta, ausencia de bucles imperativos y clasificación taxonómica de cada función. El repositorio público en GitHub refleja el proceso de desarrollo incremental mediante commits descriptivos de cada integrante del grupo.

8.2 Aprendizajes del grupo

El desarrollo de este trabajo práctico dejó aprendizajes en varios niveles. A nivel técnico, el grupo reforzó los conceptos fundamentales del paradigma funcional, como funciones puras, inmutabilidad, composición y funciones de orden superior, aplicándolos a un problema concreto del mundo real en lugar de ejercicios abstractos. La exposición a Haskell permitió contrastar dos filosofías distintas dentro del mismo paradigma: la flexibilidad dinámica de Common Lisp frente al rigor estático de Haskell, donde el compilador actúa como primera línea de defensa contra estados inválidos.



A nivel metodológico, el grupo desarrolló autonomía para aprender sintaxis y semántica de un lenguaje no visto previamente, descomponiendo el problema en funciones independientes y verificables. La adopción de Git y GitHub como herramientas de trabajo colaborativo representa un aprendizaje transversal aplicable a cualquier ámbito del desarrollo de software profesional.

8.3 Mejoras futuras

En iteraciones futuras el sistema podría extenderse en varias direcciones. A nivel de hardware, podría integrarse con plataformas como Arduino o Raspberry Pi para controlar semáforos físicos reales, convirtiendo el núcleo lógico desarrollado en un sistema embebido funcional. A nivel de persistencia, los **logs** actualmente guardados en archivo de texto plano podrían migrarse a una base de datos relacional como **SQLite**, permitiendo consultas históricas más eficientes para el equipo de auditoría forense. Finalmente, el sistema podría incorporar programación por eventos, mediante una función que se invoque automáticamente cada N segundos, detecte el cambio de estado y registre la transición sin intervención manual, acercándose al comportamiento de un sistema de control de tráfico real.



9. Bibliografía

[1] Anthropic, Claude (modelo de lenguaje de gran escala), claude.ai, 2025. [En línea]. Disponible en: <https://claude.ai>

[2] Common Lisp Documentation, Lisp Docs — Community-driven Common Lisp documentation. [En línea]. Disponible en: <https://lisp-docs.github.io/>

[3] Haskell Wiki, Aprende Haskell en 10 minutos. [En línea]. Disponible en: https://wiki.haskell.org/Aprende_Haskell_en_10_minutos

[4] B. Moure, Git y GitHub desde cero, YouTube, 2022. [En línea]. Disponible en: <https://www.youtube.com/watch?v=3GymExBkKjE>

[5] Try Haskell, Haskell Playground. [En línea]. Disponible en: <https://play.haskell.org>

[6] Quicklisp, Quicklisp — the Common Lisp package manager. [En línea]. Disponible en: <https://www.quicklisp.org>

10. Anexo

Código fuente **core.lisp**

[1] <https://github.com/MiltonRodx/TPI-Funcional-2026-Grupo-Numero31>: repositorio en GitHub

[2] **lisp/core.lisp**: implementación completa del sistema en Common Lisp

Código fuente **solucion.hs**

[1] **comparativa/solucion.hs**: re implementación de **transicion** y **semaforoTimer** en Haskell



Capturas de ejecución

```
cl-user> (run-ejemplos)
*** Requerimiento 1: ***
Camino Normal: (EN-ROJO cambiar-a-verde)
Camino alternativo: (EN-VERDE ACCION-POR-DEFECTO)%
Camino por error: (HOLA ACCION-POR-DEFECTO)
-----
*** Requerimiento 2: ***
Camino Normal: EN-VERDE
Camino alternativo: NIL%
Camino por error: NIL
-----
*** Requerimiento 3: ***
Tiempo 1970-01-03 04:33:20: la luz ha cambiado de ROJO a VERDE
Camino Normal: NIL
Tiempo 1970-08-24 22:16:34: la luz ha cambiado de en rojo a verde
Camino alternativo: NIL%
Camino por error: NIL
-----
*** Requerimiento 4a: ***
Camino Normal: (9 CICLO-LARGO)
Camino alternativo: NIL%
Camino por error: (18 CICLO-LARGO)
-----
*** Requerimiento 4b: ***
Camino Normal: CICLO-LARGO
Camino alternativo: NIL%
Camino por error: NIL
-----
*** Requerimiento 5: ***
Camino Normal: 81
Camino alternativo: NIL%
Camino por error: NIL
-----
*** Requerimiento 6: ***
Camino Normal: ((ROJO . 40.0) (AMARILLO . 2.6666667) (VERDE . 53.333336)
  (INTERMITENCIA . 2.6666667))
Camino alternativo y por error no hay (no necesita argumentos)
-----
NIL
cl-user>
```



```
≡ informe-ejecucion-semaforo.txt
```

```
1 Informe de Ejecución del Sistema Semafórico
2 =====
3 1970-01-02 07:25:23 - Transición: ROJO -> VERDE
4 1970-12-06 13:33:54 - Transición: AMARILLO -> ROJO
5 1970-01-24 09:27:12 - Transición: ROJO -> VERDE
6
7 --- Fin del Informe --- del Informe ---
```

```
cl-user> (distribucion-temporal)
((ROJO . 40.0) (AMARILLO . 2.6666667) (VERDE . 53.333336)
 (INTERMITENCIA . 2.6666667))
```

Repositorio público

[1] <https://github.com/MiltonRodx/TPI-Funcional-2026-Grupo31>

Video demo

[2] <https://www.youtube.com/watch?v=iwezgXkdh8g>